# Лабораторная работа № 4 «Диаграмма сущность-связь баз данных»

Диаграмма сущность-связь (Entity-Relationship Diagram, ERD): Эта диаграмма специально предназначена для моделирования сущностей (таких как таблицы и классы) и их отношений. В ней используются сущности (объекты) и связи между ними (отношения). ERD является частью Unified Modeling Language (UML) и предоставляет наглядное представление сущностей и их связей в системе.

*Основные элементы ERD:*

- Сущности (Entities): Сущности представляют объекты или концепции, о которых хранится информация в информационной системе. Например, в базе данных для учета студентов, сущность может представлять студента.
- Свойства (Attributes): Свойства определяют характеристики сущностей. Например, для сущности "студент" свойствами могут быть имя, фамилия, номер студенческого билета и др.
- Связи (Relationships): Связи показывают, как сущности взаимодействуют друг с другом. Они могут быть однонаправленными или двунаправленными и могут иметь атрибуты, которые описывают характер взаимодействия между сущностями. Пример связи: "студент" связан с "курсом" через связь "учится на".
- Ключи (Keys): Ключи — это атрибуты, которые уникально идентифицируют каждую сущность в базе данных. Они используются для обеспечения уникальности записей и быстрого доступа к данным.
- Служебные сущности (Associative Entities): Иногда для описания сложных связей между сущностями может потребоваться ввести дополнительную сущность, называемую служебной или ассоциативной сущностью.

*Связи*

В ERD существуют различные виды связей, которые описывают взаимодействие между сущностями в информационной системе. Основные виды связей в ERD включают:

Один ко многим:

Этот тип связи означает, что одна сущность связана с несколькими экземплярами другой сущности, но каждый экземпляр другой сущности связан только с одним экземпляром первой сущности. Например, один преподаватель может преподавать несколько предметов, но каждый предмет может иметь только одного преподавателя.

Многие к одному:

В этом случае несколько экземпляров одной сущности связаны с одним экземпляром другой сущности. Пример: несколько студентов могут быть частью одной и той же группы.

Многие ко многим:

Этот тип связи означает, что несколько экземпляров одной сущности могут быть связаны с несколькими экземплярами другой сущности и наоборот. Пример: множество студентов может изучать множество предметов.

Один к одному:

В этом типе связи каждый экземпляр одной сущности связан с одним и только одним экземпляром другой сущности и наоборот. Пример: один студент может иметь только один уникальный студенческий билет, и каждый уникальный студенческий билет принадлежит только одному студенту.

Рекомендуется избегать связей многие ко многим и один к одному в пользу связей один ко многим для упрощения и оптимизации структуры баз данных. Такой подход может быть полезным в большинстве случаев, однако это не является универсальным правилом, и необходимо учитывать конкретные требования вашей информационной системы. Вот некоторые причины, почему следует стараться использовать тип связи один ко многим:

- Упрощение структуры данных: Использование связей один ко многим может сделать структуру базы данных более понятной и легкой для администрирования и обслуживания. Однократные связи часто проще в понимании и использовании.
- Уменьшение сложности запросов: Связи многие ко многим и один к одному могут усложнять SQL-запросы и требовать более сложных JOIN-операций. Связи один ко многим позволяют более просто формировать запросы.
- Избегание избыточности данных: Связи многие ко многим могут привести к дублированию данных и увеличению размера базы данных. Связи один ко многим могут помочь избегать избыточности.
- Упрощение поддержки и модификации: Более простая структура данных обычно более легко поддерживается и модифицируется. Изменения в структуре базы данных могут потребовать меньше усилий.

Однако следует отметить, что есть случаи, когда связи многие ко многим и один к одному обоснованы и необходимы для точного моделирования бизнес-процессов. Важно анализировать конкретные требования вашей системы и принимать решение о типах связей, исходя из этих требований.

*Ключи*

Первичный ключ (Primary Key):

Первичный ключ является уникальным идентификатором каждой записи в таблице. Он используется для уникальной идентификации записи и обеспечения целостности данных. Чаще всего для первичного ключа используется вычисляемое поле с счетчиком. Иногда первичным ключом может служить естественное уникальное значение какого-либо атрибута. Например, в базе данных со студентами таким полем может быть номер студенческого. Однако следует учитывать, что в некоторых ситуациях это может создать проблемы. В приведенном примере такой проблемой может стать замена студенческого после утери.

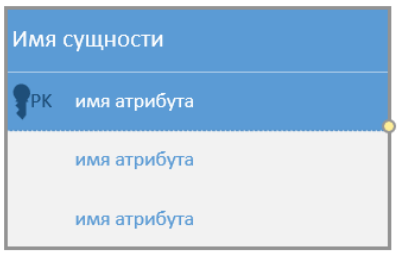
Внешний ключ (Foreign Key):

Внешний ключ представляет собой атрибут в одной таблице, который связан с первичным ключом в другой таблице. Он используется для установления связей между записями в разных таблицах. Внешние ключи обеспечивают целостность ссылочной целостности данных.

*Процесс создания ERD* обычно включает в себя следующие шаги:

1. Идентификация сущностей и их атрибутов: Определите, какие сущности существуют в вашей системе данных, и какие атрибуты они имеют.
2. Определение связей между сущностями: Определите, какие сущности связаны между собой.
3. Определение типов связей: Определите, сколько сущностей может быть связано через каждую связь.
4. Создание ERD: Нарисуйте диаграмму, представляющую сущности, атрибуты, связи и их тип.

*Графическое описание ERD:*




Сущность. В заголовке идет название сущности(таблицы), далее идут ее атрибуты. Отдельно выделяется первичный ключ.

Связи:



к одному



ко многим

**Пример процесса разработки ERD диаграммы.**

Назначение базы данных: хранение данных успеваемости студентов.

Определение Сущностей и их атрибутов в базе данных:

1) Student

   Атрибуты:

   studentID – уникальный идентификатор в базе данных, первичный ключ

   firstName – имя

   lastName – фамилия

   groupID – идентификатор группы

2) Group

   Атрибуты:

   groupID – уникальный идентификатор в базе данных, первичный ключ

   groupNumber – номер группы

   year – год обучения

3) Mark

   Атрибуты:

   markID – уникальный идентификатор в базе данных, первичный ключ

   studentID – идентификатор студента

   subjectID – идентификатор предмета

   data – дата получения оценки

   mark – оценка

4) Subject

   Атрибуты:

   subjectID – уникальный идентификатор в базе данных, первичный ключ

   subjectName – название предмета

5) Class

   Атрибуты:

   classID – уникальный идентификатор в базе данных, первичный ключ

   subjectID – идентификатор предмета

teacherID – идентификатор преподавателя

groupID – идентификатор группы

6) Teacher

Атрибуты:

teacherID – уникальный идентификатор в базе данных, первичный ключ

firstName – имя

lastName – фамилия

teachingPosition – должность

Определение связей:

Один *студент* может иметь множество *отметок*;

В одной *группе* учится множество *студентов*;

По одному *предмету* можно получить множество *отметок*;

По одному *предмету* может быть множество *занятий*;

У одной *группы* может быть множество *занятий*;

Один *преподаватель* может вести несколько *занятий*;

Сущность Class(занятие) в данной базе данных нужна для упрощения структуры. Она позволяет избежать создания связей многие ко многим между предметом и отметкой. В данной базе данных эта сущность является служебной.

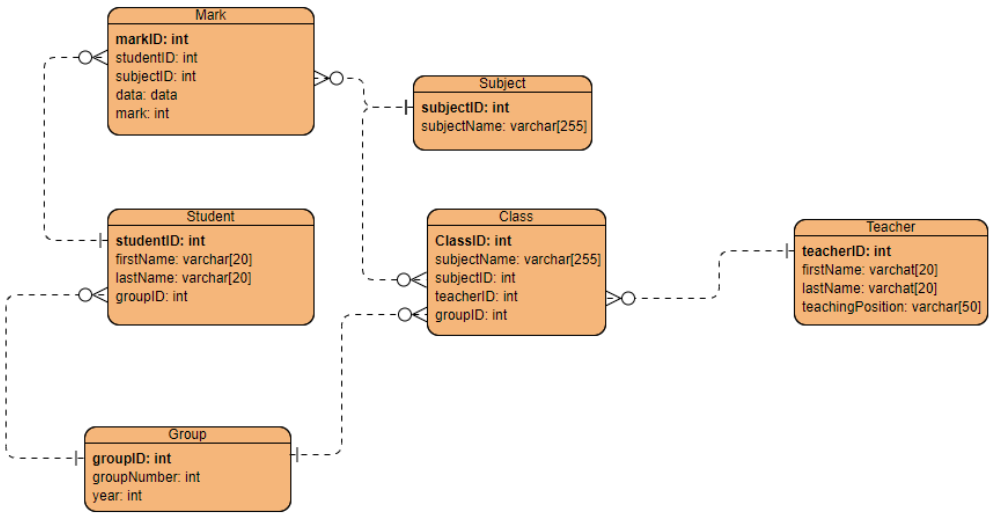



Рис. 1 – пример диаграммы

**Задание:**

1) Изучить теоретические материал
2) В соответствии с темой прошлой работы описать структуру базы данных;
3) Разработать ER-диаграмму описанной базы данных;
4) Оформить отчет.

**Контрольные вопросы:**

1) Что такое ERD (диаграмма сущность-связь) и для чего она используется?
2) Какие основные элементы ERD, как они представляются на диаграмме?
3) Зачем нужен первичный ключ?
4) Назовите типы связей между сущностями.